Lab No 2: **Date:**

OpenMP programs on work-sharing constructs

Objectives:

In this lab, student will be able to

At the end of this laboratory session, students will be able to:

1. Understand the concept of work sharing in OpenMP.
2. Explain the purpose of OpenMP work-sharing constructs.
3. Implement parallel loops using the parallel for directive.
4. Use the sections construct to assign different tasks to different threads.
5. Use the single construct to execute a code block by only one thread.
6. Observe how OpenMP distributes work among threads using omp_get_thread_num().

Work-Sharing in OpenMP

A **work-sharing construct** divides the execution of a block of code among the threads that already exist in a parallel region. Unlike a simple parallel construct, where every thread executes the same block of code, work-sharing constructs ensure that each thread performs only a portion of the total work.

Work-sharing improves efficiency by eliminating redundant computation and distributing independent tasks across multiple threads.

Characteristics

- Work-sharing constructs **do not create new threads**.
- They distribute work among the threads already present in the parallel region.
- There is **no implicit barrier at the beginning**.
- By default, there is an **implicit barrier at the end** of every work-sharing construct (unless the nowait clause is specified).

Common Work-Sharing Constructs

OpenMP provides the following work-sharing constructs:

1. for / parallel for

Distributes loop iterations among available threads.

```
#pragma omp parallel for
for(int i=0;i<N;i++)
{
   // loop body
}
```

2. sections

Distributes independent blocks of code among different threads.

```
#pragma omp parallel sections
{
   #pragma omp section
   {
      // Task 1
   }

   #pragma omp section
   {
```

```
        // Task 2
    }

    #pragma omp section
    {
        // Task 3
    }
}
```

Each section is executed by one thread.

3. single

Ensures that only one thread executes a particular block.

```
#pragma omp parallel
{
    #pragma omp single
    {
        printf("Executed by one thread.\n");
    }
}
```

When is Work Sharing Useful?

Work sharing is beneficial when:

- Loop iterations are independent.
- Array processing is performed.
- Matrix operations are executed.
- Numerical computations involve repetitive calculations.
- Image processing operations are independent.
- Scientific simulations contain independent tasks.

Example Program

Array Addition using parallel for

```
#include <stdio.h>
#include <omp.h>
#define N 10

int main()
{
    int a[N], b[N], c[N];
    int i;

    for(i=0;i<N;i++)
    {
        a[i]=i;
        b[i]=2*i;
    }

    #pragma omp parallel for
    for(i=0;i<N;i++)
    {
```

```
        c[i]=a[i]+b[i];

        printf("Thread %d computed c[%d] = %d\n",
            omp_get_thread_num(), i, c[i]);
    }

    return 0;
}
```

Compile

```
gcc array_add.c -fopenmp -o array_add
```

Run

```
./array_add
```

Laboratory Exercises

1. Implement an OpenMP program using the parallel for work-sharing construct to perform the addition of two matrices of size M×N. Display the resultant matrix and the Thread ID responsible for computing each row. Compare the execution time of the serial and parallel implementations and comment on the distribution of loop iterations among the threads.
2. Implement an OpenMP program to read a matrix A of size 5×5 and produce matrix B according to the specified transformation:
   - Principal diagonal = 0
   - Elements below the diagonal = maximum value of the corresponding row in A
   - Elements above the diagonal = minimum value of the corresponding row in A

   Display both matrices and indicate the Thread ID responsible for processing each row.
3. Implement an OpenMP program that reads a matrix of size $M \times N$ and produces:
   - Matrix B, where all non-border elements are replaced by their 1's complement.
   - Matrix D as specified.

Matrix A

1	2	3	4
6	5	8	3
2	4	10	1
9	1	2	5

Matrix B

1	2	3	4
6	**10**	**111**	3
2	**11**	**101**	1
9	1	2	5

Matrix D

1	2	3	4
6	**2**	**7**	3
2	**3**	**5**	1
9	1	2	5

4. Implement an OpenMP program to perform matrix-vector multiplication. Record the effect of increasing matrix size on execution time.
5. Implement an OpenMP program to generate an array of random integers using the single construct. Then use the parallel for directive to compute the square of each element. Display the Thread ID that generates the array and the Thread IDs processing the elements.
6. Implement a parallel program using OpenMP to perform vector addition, subtraction, multiplication. Demonstrate task level parallelism. Analyze the speedup and efficiency of the parallelized code.

Performance Analysis

1. Measure the execution time using omp_get_wtime().
2. Compare sequential and parallel execution times.
3. Compute:
   - Speedup
   - Efficiency
4. Study the effect of varying the number of threads using:
   export OMP_NUM_THREADS=2
   export OMP_NUM_THREADS=4
   export OMP_NUM_THREADS=8

Additional Exercise:

1. Implement an OpenMP program to calculate $pow(i, x)$, where i is an integer entered by the user. $x$is the Thread ID.
2. Implement an OpenMP program that performs the addition of two one-dimensional arrays using the **parallel for** directive.
3. Implement an OpenMP program to reverse the digits of every element in the following array:
   18, 523, 301, 1234, 2, 14, 108, 150, 1928
   Output: 81, 325, 103, 4321, 2, 41, 801, 51, 8291
4. Implement an OpenMP program to generate prime numbers within a user-specified interval using the **parallel for** directive.